TEN1 HI1027 2024-10-21, lösningsförslag och rättningsmall

A-del

Kommentarer till rättningen anges inom [...]

1. B. Att dölja den interna implementationen i klassen och utåt, dvs. för andra klasser, endast exponera den nödvändiga funktionaliteten.
   D. Att säkerställa att objekten i alla lägen följer kontraktet för den aktuella typen (klassen).
   [Endast B eller endast D: 1 p]

2. A. Abstrakta metoder
   B. Metoder med implementation
   C. Datamedlemmar (instansvariabler)
   D. Konstruktorer
   [Ett alternativ saknas: 1 p]

3. B. Hög kohesion syftar på att implementationen av en klass, och dess metoder, fokuserar på att abstrahera en enda roll eller funktionalitet i applikationen.
   [Endast ett svarsalternativ är korrekt, fler än ett svar ger 0 p]

4. B. I aggregation kan de ingående objekten existera oberoende av varandra, medan i komposition kan inte delarna existera utan det överordnade objektet.
   [Endast ett svarsalternativ är korrekt, fler än ett svar ger 0 p]

5. a) ImageProxy implementerar interfacet Image, dvs. ger en implementation av varje abstrakt metod från interfacet.
   b) ImageProxy är ett aggregat av ett objekt av typen ImageFile, dvs. ImageProxy har en referens av typen ImageFile som datamedlem.
   [Endast namn på relation räcker inte för full poäng]

6. Med try-finally:
```
public String  openAndRead(File file) throws IOException {
   BufferedReader reader = null;
   try {
      reader = new BufferedReader(…);
      String text = reader.readLine();
      // … read more …
      return text;
   }
   finally {
      if(reader != null) reader.close();
   }
}
```
   Alternativt, med “try-with resources”:
```
public String  openAndRead(File file) throws IOException {
   BufferedReader reader = null;
   try(BufferedReader reader = new BufferedReader(…)) {
      String text = reader.readLine();
      // … read more …
      return text;
   }
}
```

[Eftersom metoden inte ska hantera felet ska det inte finnas något catch, och metoden måste då märkas med ”throws” eftersom IOException är ”checked”. ”catch” (utan att exception kastas vidare) ger 0 p. Ett catch som kastar ett nytt exception är poänglöst om inte det nya undantaget bättre visar vad som gick fel: -1p
Med try-finally är det viktigt att referensen reader deklareras före try-finally så att den är ”känd” i båda kodblocken, samt att kontroll mot null sker i finally-blocket.
Metodsignatur saknas (inget throws): -1p]

7. Nyckelordet synchronized markerar ett kodblock som endast en tråd i taget får exekvera, kodblocket exekveras i sin helhet av en tråd innan nästa tråd får exekvera blocket. För att kunna samsynkronisera flera kodblock anges en referens till ett objekt vid kodblocket (lås/monitor). Poängen med synkroniserade kodblock är att skydda delat data mot problem som kan uppstå då flera trådar vill läsa/skriva till detta data. Exempel med delat konto, där access till balance ska synkroniseras:
```
Account a = new Account(…);
…
synchronized(a) { a.withdraw(...); }
…
synchronized(a) { a.deposit(…); }
```
(Kommentar: Att ett kodblock/metod är synkroniserat innebär inte per automatik att metoden är trådsäker – data som ska skyddas kanske manipuleras på andra ställen i koden.)
[För full poäng måste det framgå att det är kodblock eller metoder som synkroniseras, men att detta görs för att skydda data, samt ett rimligt kodblock som visar syntax med användning av monitor/lås presenteras.]

8. Designmönstret Singleton används då man vill garantera att endast ett objekt, av en viss typ, kan skapas och att detta enda objekt ska vara tillgängligt överallt. Man ”gömmer” då konstruktorn, genom att göra den privat och skriver en publik klassmetod för att accessa det enda objekt – metoden skapar objekt om det inte redan existerar.
```
public class Singleton {
   private static Singleton theObject; // reference to the single object
   private Singelton(…) { … } // private construktor
   public static Singleton() getInstance() {
      if(theObject == null) theObject = new Singleton(…);
      return theObject;
   }
   …
}
```
[Konstruktorn måste vara privat – notera att om ingen konstruktor deklareras fås en publik parameterlös konstruktor.
Metoden getInstance måste deklareras static (ska kunna anropas utan objekt-referens) och endast skapa objektet om detta inte redan existerar (theObject == null). Även theObject måste deklareras static, akn annars inte manipuleras i static-metoden getInstance.]

B-del

För godkänt på denna del måste 9A eller 10A vara helt korrekt och sammanlagda poängen på B-delen vara minst 8 poäng.

Lösningsförslag finns på https://gits-15.sys.kth.se/anderslm/HI1027-TEN1

För alla uppgifter på B-del: get/set-metoder som exponerar referenser till privat data: -1p – *oftast via autogenererade get/set-metoder – måste kontrollera autogenererad kod.*

9. A (4 p)
   Om AShape implementerar move (flyttar point.x och point y) behöver inte Circle omdefiniera metoden, däremot måste Line omdefiniera move så att båda punkterna flyttas.
   Om draw inte skriver ut information om ärvt data (använd accessmetoder från AShape): -2p
   Subklasskonstruktor initierar inte ärvt dat: -1p
   Deklarerar redan ärvt data på nytt i subklass: -4p (ej förstått arv)

   B (2 p)
   Interna listan och metoden add (lägga till IShape) ska ha typparameter respektive argument av typen IShape, så att alla typer i arvshierarkin kan läggas i en Group.
   Använder instanceof i Group (har ju dynamisk bindning vid anrop av mode/draw): -1p

10. A (4 p)
    Konstruktor allokerar inte egen intern char-array: -1p
    Notera att datamedlem för antal tecken är redundant, använd chars.length.
    B (2 p)
    Använder inte den generiska versionen av Comparable: -1p

11. Ej generisk lösning eller använder lista (istället för array) internt: -3p
    Korrekt generisk lista förutom forEach (och interface) ger 2p